

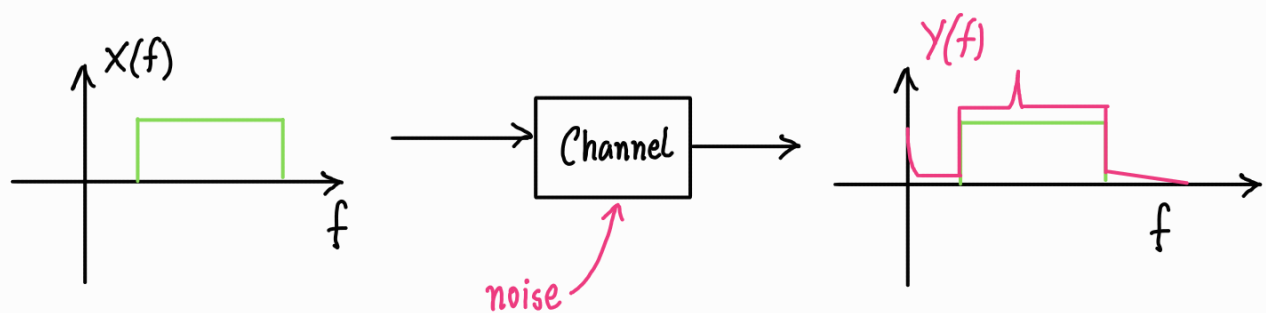
Intro to filters —

We generally see a signal as some values in time. But there is a whole different domain depending on how fast the signal changes — the frequency domain. We will learn a lot about the frequency domain after mid. For now we will keep things limited.

What we should know for now is that any meaningful signal that we encounter in real life is composed of different frequencies.



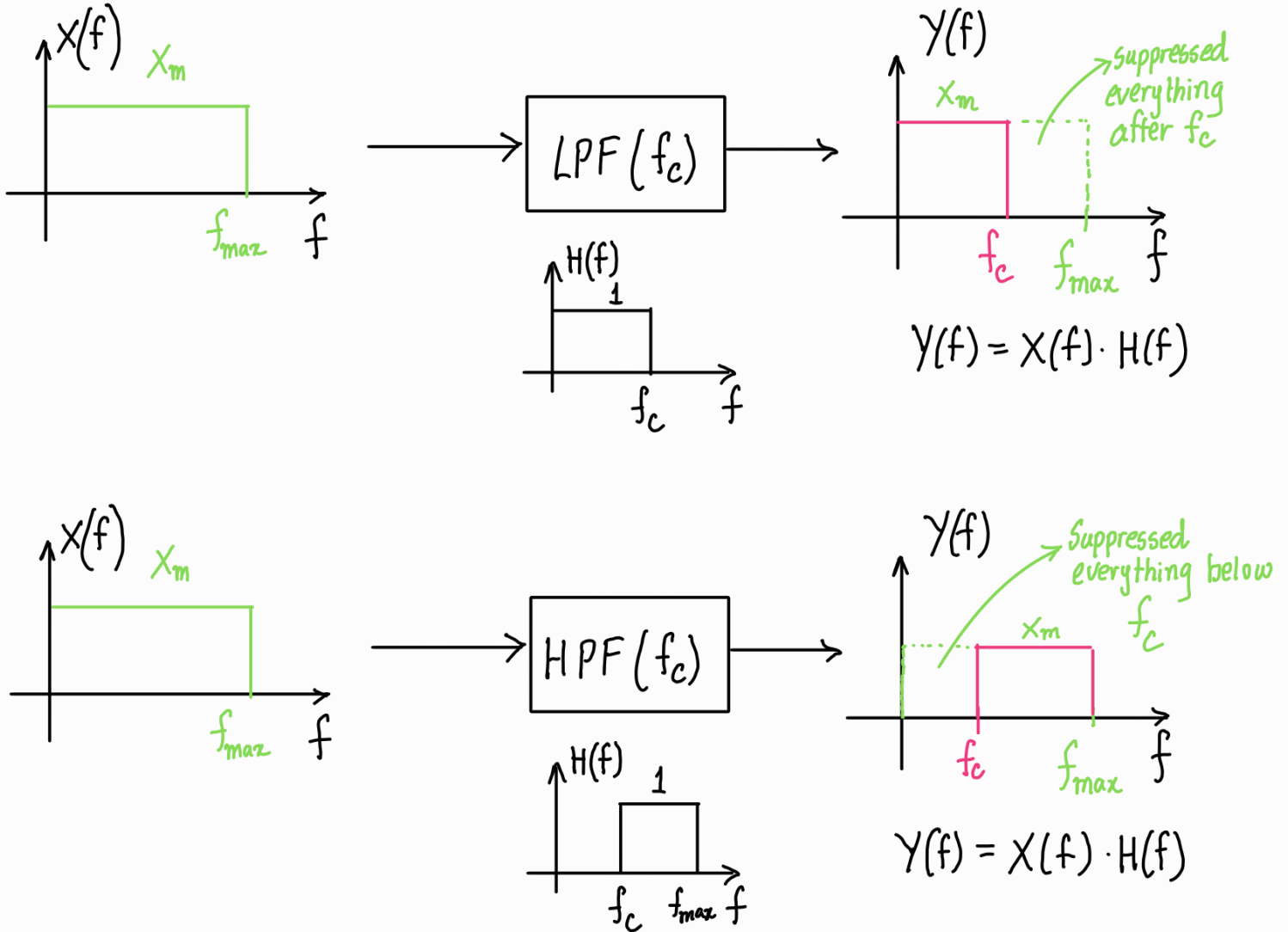
Now assume you are sending information over a communication channel and the channel adds some noise over the signal —



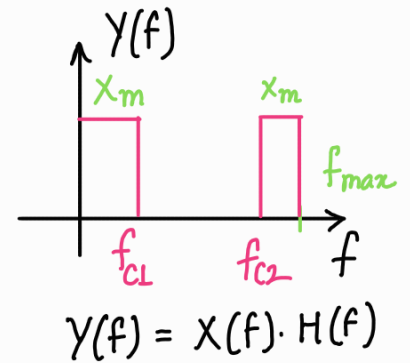
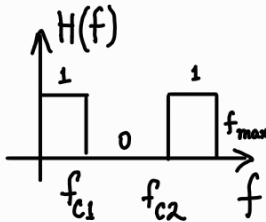
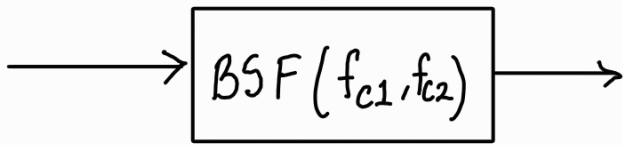
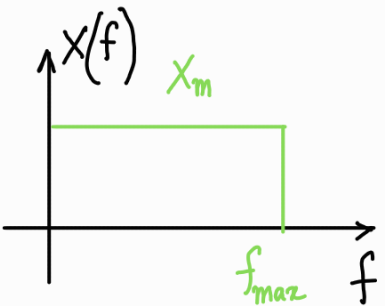
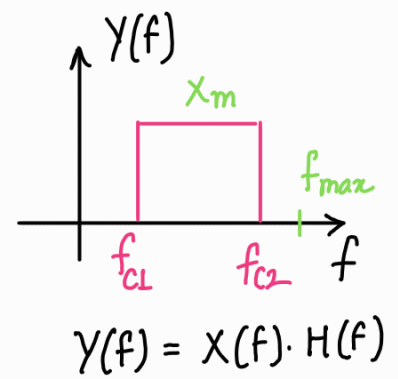
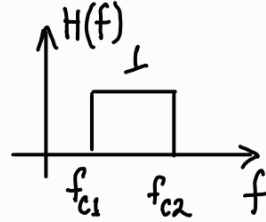
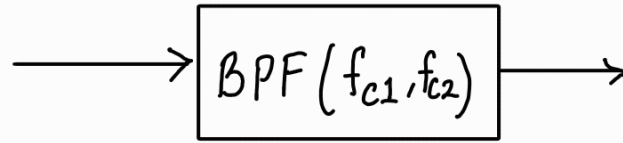
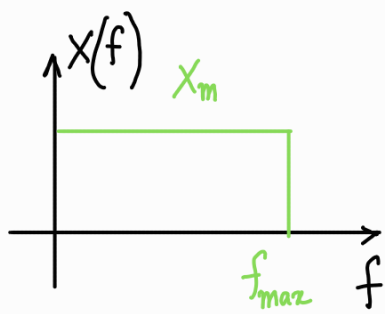
How do we fix this? We filter our output to fix it.

What is a filter. Conventionally filters suppress a certain part of the bandwidth and let the rest pass.

The most common types of filters are — (i) Low pass filter (LPF) → Suppresses high frequencies
(ii) High pass filter (HPF) → Suppresses low frequencies



We also have composite filters like — (i) Bandpass filter (BPF) ← Passes a certain frequency range
(ii) Bandstop filter (BSF) ← Suppresses a certain frequency range



I mentioned that these are composite filters because a $BPF(f_{c1}, f_{c2}) = LPF(f_{c2}) \cdot HPF(f_{c1})$ and a $BSF(f_{c1}, f_{c2}) = LPF(f_{c1}) + HPF(f_{c2})$

We will learn more about filters later but for now we will go back to convolution.

Convolution and filtering —

There is a relation between frequency and time domain. Each filter in frequency domain is a system, but the system is defined over frequency. Now the magic transformation can bring $H(f)$ to $h(t)/h[n]$. But whatever is multiplication in frequency domain is Convolution in time domain. So,

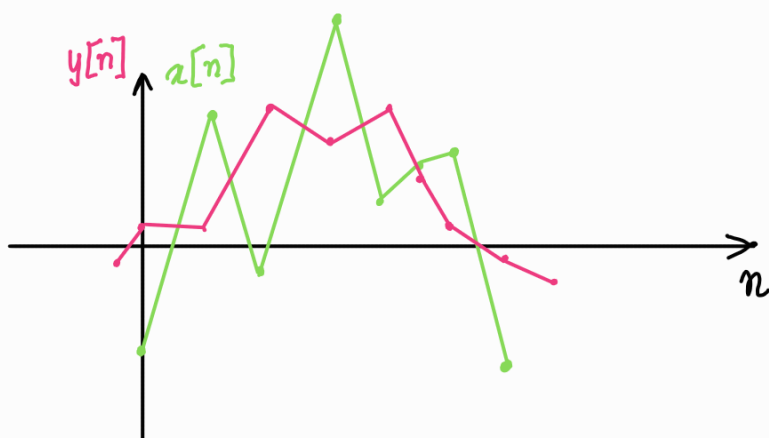
$$y[n] = h[n] * x[n] \iff Y(f) = H(f) \cdot X(f)$$

But designing in time domain is very hard and unintuitive. So we don't do that, at least for 1D signals. But it's still possible nonetheless. So we will see two examples for 1D signals. We will see an LPF and an HPF filter in action. We don't need the math tbh now. We need the concept and we will extrapolate it to CNNs over the next two lectures.

A very simple LPF filter is a moving average system, $h[n] = \frac{1}{3} \cdot \{1, \underset{\uparrow}{1}, 1\}$ origin set at the middle generally

Assume $x[n] = \{-14, 26, -2, 54, 7, 15, 17, -21\}$ ← fast change indicates high frequency presence

$y[n] = \{-4.67, \underset{\uparrow}{4}, 3.33, 26, 19.67, 25.33, 13, 3.67, -1.3, -7\}$ ← the difference between each sample is lower than before now



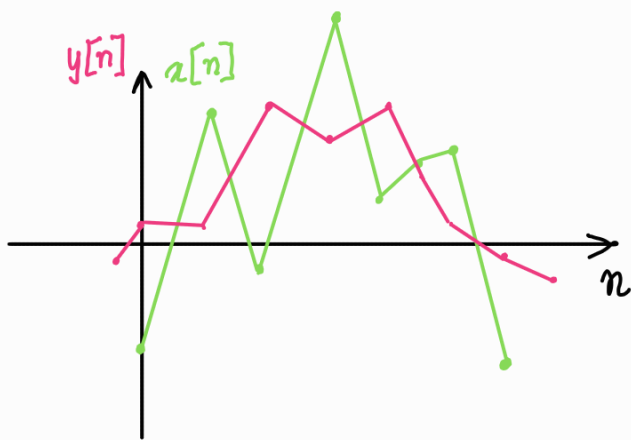
A generalized filter is often referred to as a kernel in computer vision lingo and from now on we will use the term kernel interchangeably with filter.

Now as for an HPF, a simple implementation maybe a difference filter,

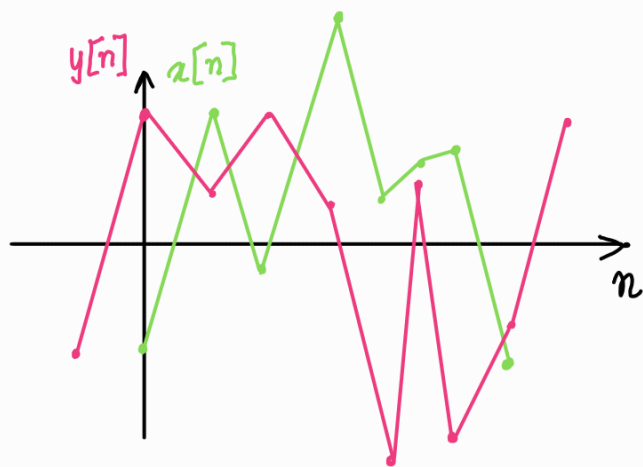
$$h[n] = \{1, \underset{\uparrow}{0}, -1\}$$

high difference between each sample, the flat region is basically lost

then, $y[n] = x[n] * h[n] = \{-14, 26, \underset{\uparrow}{12}, 24, 9, -39, 10, -36, -17, 21\}$



After LPF



After HPF

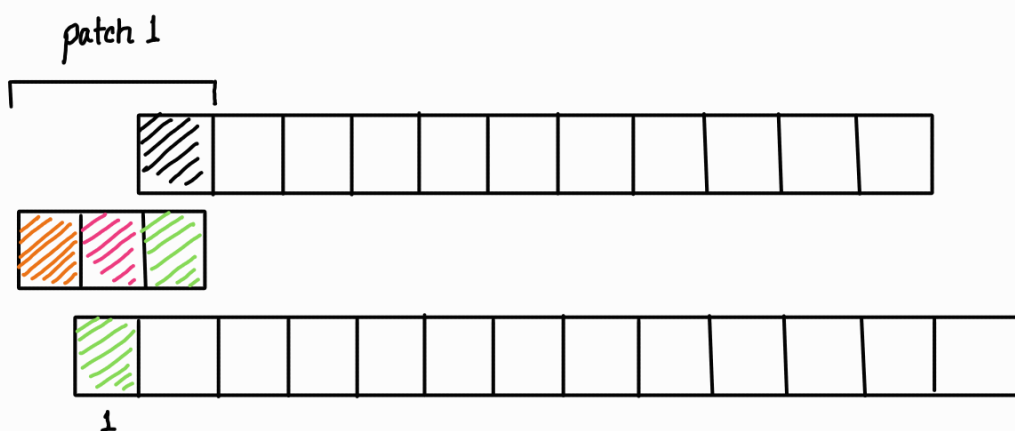
Now they do look a bit shifted after filtering and its a normal phenomenon. But the intuition stands. In the lab segment you will get to see it better for longer signals along with the frequency domain changes.

The sliding kernel and the edges —

kernel — 

Reversed kernel — 

Signal — 

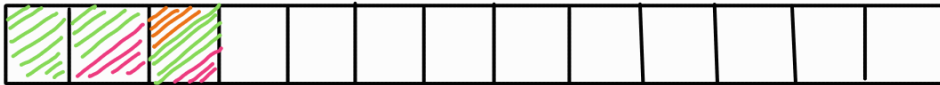
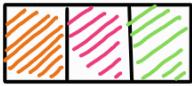
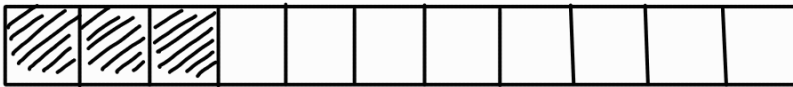


patch 2



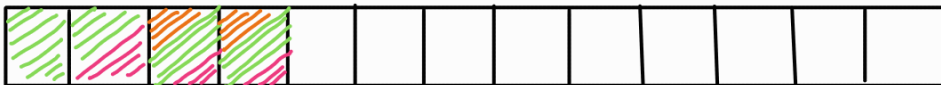
1 2

patch 3



1 2 3

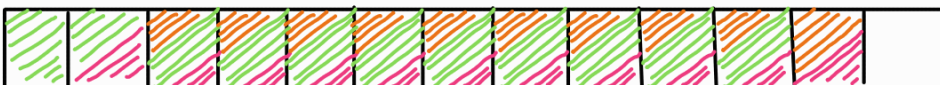
patch 4



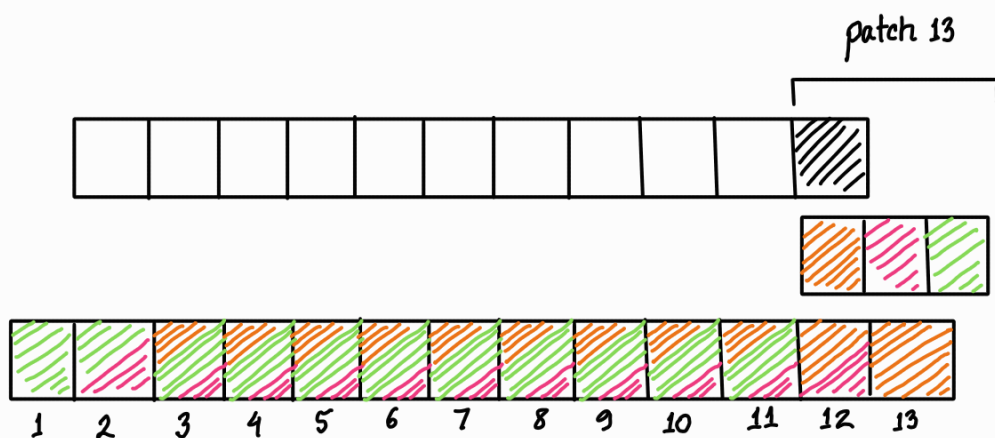
1 2 3 4

•
•
•

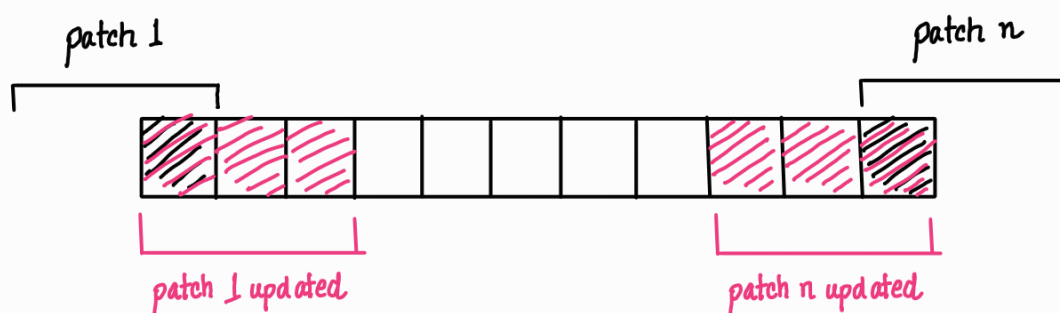
patch 12



1 2 3 4 5 6 7 8 9 10 11 12

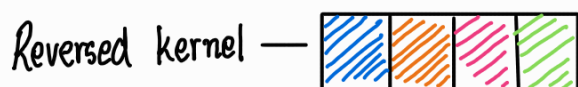


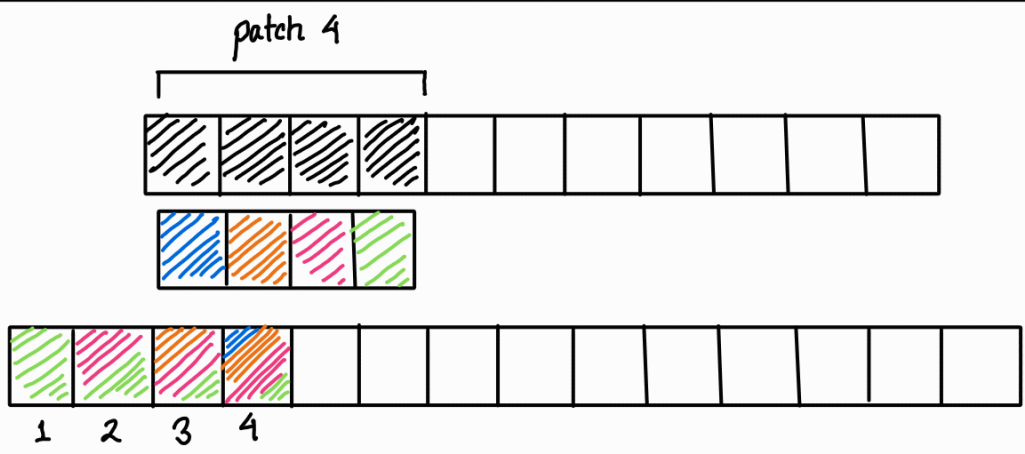
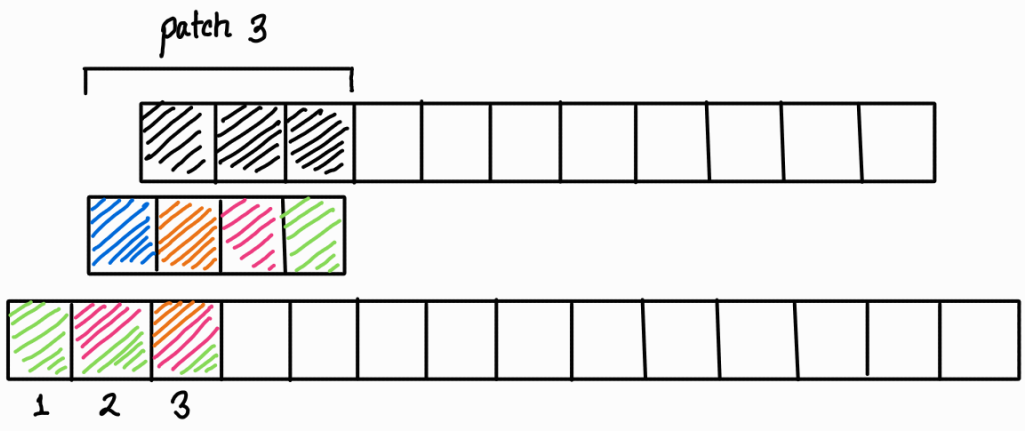
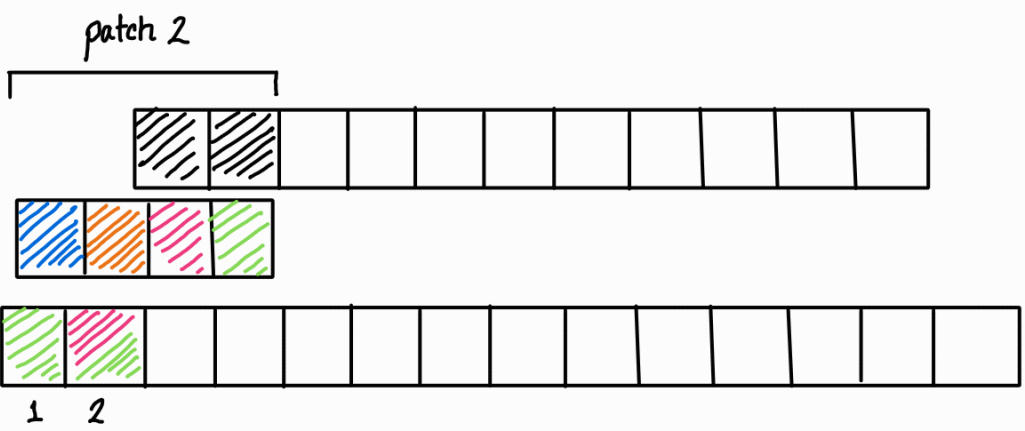
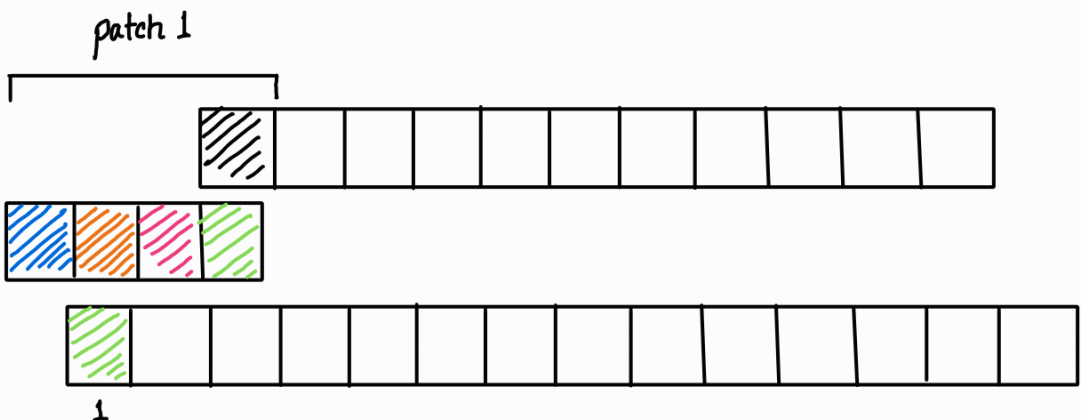
Here notice how the patches 1, 2, 12 & 13 contain incomplete kernel information. This is because they sit on the edges. So while applying filters, we generally start from full overlap region and avoid the edges. The built in convolve functions follow this rule. In case you are coding from scratch you just have to ensure that you are accounting for the kernel size. Also from here on we will not be bothered too much about the time values as we go through CNNs. Also notice how we are losing some dimension for omitting the edges. We are losing exactly $(\text{kernel size} - 1)$ dimensions. From now onwards, at least for the CNN portion —



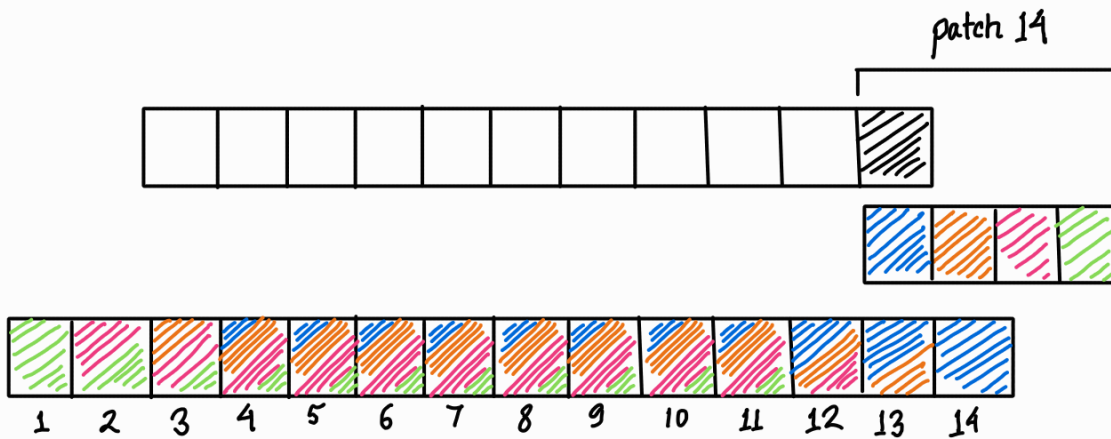
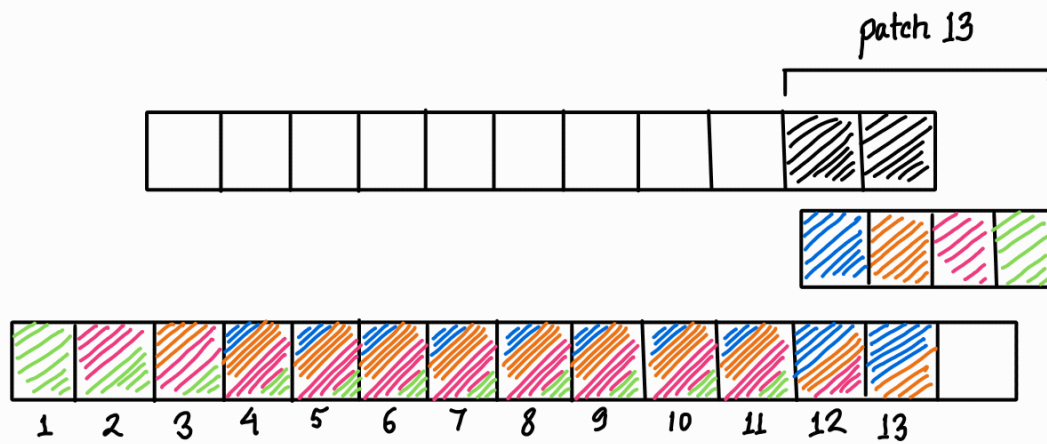
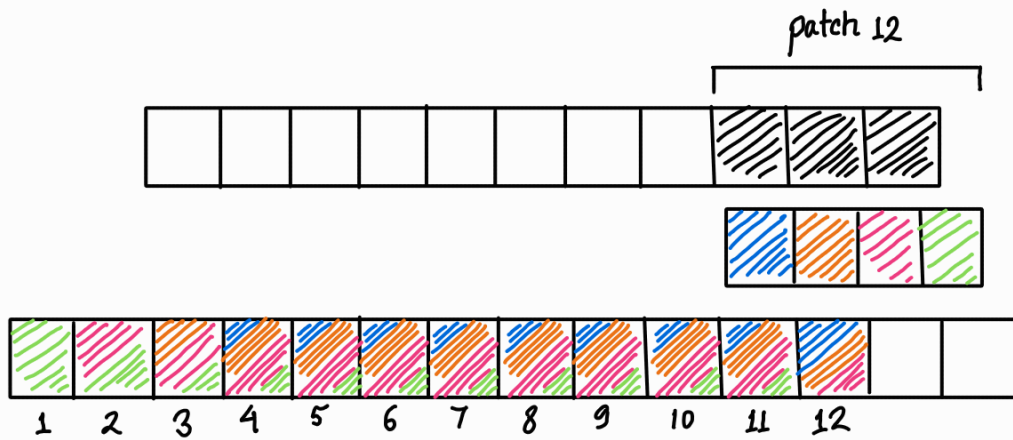
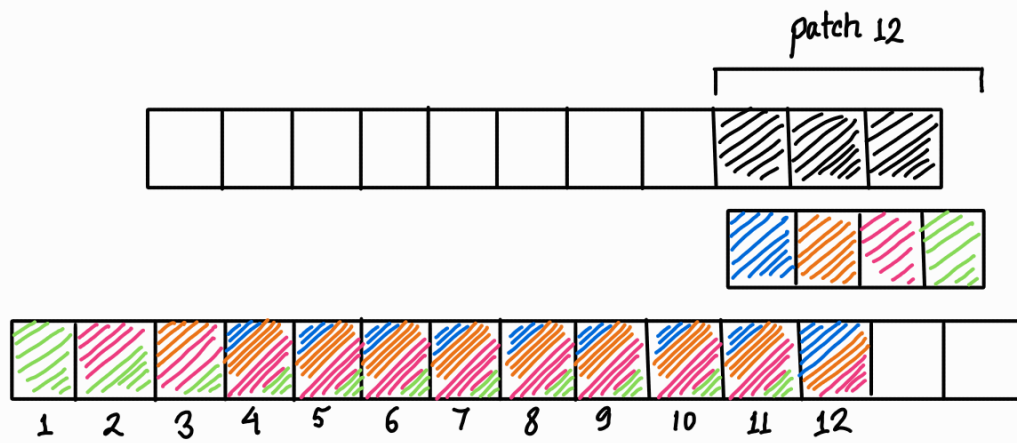
For demarcation purposes we will call this edge truncated convolution or truncated convolution.

Now let's see the same thing for an even sized kernel —





•
•
•



We reach the same conclusion —

$L_{out} = L_{in} + k - 1$ for regular convolution

$L_{out} = L_{in} - k + 1$ for truncated convolution

In the next lecture we will see filters for 2D signals, the sliding operation for 2D signals, and implications of filtering on signals.